

Topic 5.2

5.2: Conversions Between a Basic Type and a User-defined Type

To permit conversion from a basic type to a user-defined type, the user-defined type must specify either a constructor with a single argument of the correct type, or an overloaded `operator=` whose source is of the appropriate type. The compiler first looks for the overloaded `operator=`. Failing to find one, it will then look for an appropriate constructor.

As an example, suppose we wanted to be able to assign `float` values to our `Distance` variables, where the `float` value represents metric distance in meters. The most common approach is to create a new constructor:

```
class Distance
{
    ...
    Distance( float n_meters );
    ...
};

Distance::Distance( float n_meters )
{
    double total_feet;

    total_feet = n_meters * FEET_PER_METER;
    feet = (int)(floor(total_feet));
    inches = (total_feet - feet) * 12;
}
```

If we wish to convert the other direction, from a user-defined type to a basic type, we must overload a cast operator:

```
class Distance
{
    ...
    operator float(); // Note: no return type
                      // may be specified.
    ...
};

Distance::operator float()
{
    return (float)((feet + inches/12) / FEET_PER_METER);
}
```

This is only a code fragment to illustrate this particular concept. The following complete program includes the code fragment:

```
// MODULE:      distanc5.cpp
// PROGRAMMER: Gary J. Blair
// DATE:        96/04/11
// COMPILER:    MS Visual C++ 2008 Express Edition
// REQUIRED:     distance5.cpp, dist5.h, dist5.cpp
// PURPOSE:
//   Provide a class for English distances.
//   Illustrate conversions.

#include <iostream>
#include <stdlib.h>
#include "dist5.h"

using namespace std;

int main(void)
{
    Distance  a;
    Distance  b(5.2);    // Initialize b
                        // to 5.2 meters.

    Distance  c;
    float     x;

    a = 2.5;             // Constructor called
                        // for conversion.

    cout << "2.5m = " << a << endl;
    cout << "5.2m = " << b << endl;

    // Explicit use of conversion constructor.
    c = Distance(3.8) + b;
    cout << "c = 3.8m + " << b << " = " << c
         << endl;

    x = c;
    cout << c << " = " << x << "m\n";
    c = x;
    cout << "After converting to metric then back
            to English c = " << c << endl;

    return 0;
}
```

Output for distanc5.cpp:

```
2.5m = 8'2.42499"
5.2m = 17'0.723972"
c = 3.8m + 17'0.723972" = 29'6.32995"
29'6.32995" = 9m
After converting to metric then back to English c = 29'6.32996"
```

```
// MODULE:      dist5.h
// PROGRAMMER: Gary J. Blair
// DATE:        4/11/1996
// COMPILER:    MS Visual C++ 2008 Express Edition
// REQUIRED:     distanc5.cpp, dist5.h, dist5.cpp
// PURPOSE:
```

```

// Header file for Distance class

using namespace std;

class Distance
{
    public:

    // The no argument constructor
    // initializes a Distance to 0'0".

    Distance();

    // Initialize a Distance to the specified value.

    Distance( int ft, double in );

        // Initialize a Distance to the specified
        // value, which is in meters. This can
        // also be used as a conversion operator.

    Distance( float n_meters );

        // Convert English Distance to meters.

    operator float(); // Note: no return type
                     // may be specified.

        // Define + that operates on
        // Distance + Distance.

    Distance operator + ( Distance &rhs ) const;

        // Define stream insertion operator
        // for Distances.

    friend ostream& operator << ( ostream& str,
                                   Distance& d );

        // Define stream extraction operator
        // for Distances.

    friend istream& operator >> ( istream& str,
                                   Distance& d );

    private:
    int      feet;
    double   inches;
};

// MODULE:      dist5.cpp
// PROGRAMMER:  Gary J. Blair
// DATE:        4/11/1996
// COMPILER:    MS Visual C++ 2008 Express Edition
// REQUIRED:     distanc5.cpp, dist5.h, dist5.cpp
// PURPOSE:
// Implement the Distance class.

#include <iostream>
#include <stdlib.h>
#include <math.h>
#include "dist5.h"

#define FEET_PER_METER 3.280833

    // The no argument constructor
    // initializes a Distance to 0'0".

```

```

Distance::Distance()
{
    feet = 0;
    inches = 0.0;
}

// Initialize a Distance to the specified value.

Distance::Distance( int ft, double in )
{
    feet    = ft;
    inches  = in;
}

// Initialize a Distance to the specified value,
// which is in meters.
// This can also be used as a conversion
// operator.

Distance::Distance( float n_meters )
{
    double total_feet;

    total_feet = n_meters * FEET_PER_METER;
    feet = (int)(floor(total_feet));
    inches = (total_feet - feet) * 12;
}

// Convert English Distance to meters.

Distance::operator float()
{
    return (float)
        ((feet + inches/12) / FEET_PER_METER);
}

// Define + that operates on
// Distance + Distance.

Distance Distance::operator + ( Distance &rhs )
const
{
    int    sum_feet;
    double sum_inches;

    sum_feet = feet + rhs.feet;
    sum_inches = inches + rhs.inches;
    if (sum_inches >= 12.0)
    {
        sum_inches -= 12.0;
        ++sum_feet;
    }
    return Distance(sum_feet, sum_inches);
}

// Define stream insertion operator
// for Distances.

ostream& operator << ( ostream& str, Distance& d )
{
    str << d.feet << " " << d.inches << " \n";
    return str;
}

// Define stream extraction operator
// for Distances.

```

```
istream& operator >> ( istream& str, Distance& d )
{
    str >> d.feet;
    if (str.get() != '\\')
    {
        cerr << "*** Error extracting Distance.\n";
        exit(1);
    }
    str >> d.inches;
    if (str.get() != '')
    {
        cerr << "*** Error extracting Distance.\n";
        exit(1);
    }
    return str;
}
```

Topic 5.3 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-5-dot-3>)